Activity 4: Effect of Sample Size on Statistical Timing-Leakage Detection

CS6530 Applied Cryptography – Theme 13

1 Objective and hypothesis

Activity 4 asks for one controlled analytical parameter to be changed, a hypothesis to be stated first, and the change in confidence in the leakage conclusion to be explained. The parameter changed here is the *sample size*, the number of timing observations per class. Everything else (channel, analysis method, α, bootstrap repetitions, machine) is held fixed.

Hypothesis (stated before running). For a small but real timing difference, the RTLF decision rule will usually report no leak at a small sample size and will report it more often as the sample size grows. For a channel with no real difference, detections should stay rare (about α or below) at every size. More data should therefore raise confidence in a leakage conclusion without raising the false-alarm rate.

This follows the primary paper: for the artificial `shift-1%` channel, Table 2 reports a detection rate of 22.6% at 30k samples and 97.5% at 500k. Table 3 shows the type-1 error of RTLF staying within the configured α.

2 What the experiment does

2.1 Data source

The full Zenodo dataset is 38.7 GB, so it was not downloaded. The study uses only the authors' 15.7 kB archive `f_source_files_artificial_side_channels.zip`, which contains a C client and server for each artificial side channel of the paper's Table 1. The measurements are collected locally.

- `shift-1%` (leaking): the two classes BASELINE and MODIFIED differ by a small fixed shift (about 1% of the executed busy-wait work).
- `same-xy` (negative control): both classes are generated identically, so any detection is a false positive.

2.2 Measurement procedure

The author-provided client is patched so that the number of observations is a command-line argument instead of the fixed value of 10^6 (a student modification). For each run the Python driver starts the server on local TCP port 8080 and runs the client. The client sends requests of the two classes in randomized order and records the round-trip time of each request with the x86 cycle counter (`RDTSC`). The driver stores one CSV (`class, cycles`) per run and checks that the two classes are balanced.

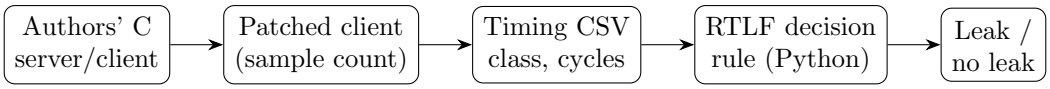


Figure 1: Pipeline of the experiment script.

2.3 Statistical analysis

The analysis is a Python implementation of the decision rule of the primary paper (Algorithms 1 and 2), not the authors' R tool itself.

1. For both classes the nine deciles $(10\%, \ldots, 90\%)$ are computed with the type-2 quantile estimator, and the observed difference $\hat{Q}_j = |\hat{q}_j^X - \hat{q}_j^Y|$ is taken for each decile j.
2. The expected noise per decile is estimated by a bootstrap. Each class is resampled twice, and the absolute difference between the two resampled deciles is recorded. This is repeated $B = 10{,}000$ times.
3. The threshold c_j^* is the $(1 - \alpha/9)$ quantile of the bootstrap differences, taking the larger value of the two classes. The factor $1/9$ is the Bonferroni correction for nine tests.
4. H_0 (equal distributions) is rejected if $\hat{Q}_j > c_j^*$ for *at least one* decile. The test is run with $\alpha = 0.09$.

For speed, the bootstrap quantiles are drawn directly from Beta order-statistic distributions instead of materializing 4×10^4 resamples of up to 5×10^5 values each. This is a computational shortcut for the same bootstrap distribution. Each run's bootstrap seed is a hash of its file name, so every analysis is reproducible.

2.4 Design

Parameter	Value
Varied	observations per class: 30,000 / 200,000 / 500,000
Channels	`shift-1%` (leak), `same-xy` (control)
Independent collection runs per cell	3
α, bootstrap repetitions B	0.09, 10,000
Held fixed	machine, tool code, load conditions as far as possible

3 Configuration

Item	Value
Machine	Lenovo IdeaPad Slim 5 16IAH8, x86_64
CPU model	`12th Gen Intel(R) Core(TM) i5-12450H`
Python	3.12.3
Compiler	GCC 13.3.0 , flag `-O2`
Dataset	authors' source archive, MD5 `c74935b02c68e922de1cd8f5b9906f6e`

The 30k and 200k sizes come from one command (`activity4_results/proof_run`); the 500k size was run separately (`activity4_results/n500k`) with identical settings.

4 Results

Table 1: RTLF detections out of 3 independent runs per cell ($\alpha = 0.09$, $B = 10{,}000$). Deciles in parentheses are those exceeding their threshold.

Channel	30k per class	200k per class	500k per class
`shift-1%` (leak)	0/3	1/3 (20, 30)	2/3 (10, 10)
`same-xy` (control)	0/3	1/3 (30)	0/3

Baseline. At 30k per class RTLF reported no difference in any of the three runs, for either channel. For `shift-1%` this matches the paper's low small-sample power (22.6%): with that detection rate, 0 of 3 detections would occur by chance about 46% of the time (0.774^3).

Modified setting. For the leaking channel the number of detections rises with sample size, from 0/3 to 1/3 to 2/3. At 500k both detections were in the 10th decile only. A possible explanation is that the fast end of the distribution has the least noise, so a small constant shift stands out there first. This is a hypothesis and was not tested.

Control. Across the nine control runs there was one detection (200k, decile 30). With $\alpha = 0.09$, about 0.8 false alarms are expected in nine runs, and the chance of at least one is about 57%, so a single detection is compatible with the configured error rate. The control had no detections at 500k.

5 Interpretation: how confidence changes

- **Direction agrees with the paper.** Detections of the real 1% shift increase with sample size, and the control stays mostly clean. This is the pattern predicted by the primary paper, where statistical power approaches 1 as n grows while the type-1 error remains bounded.
- **The evidence is weak.** With three runs per cell, the data cannot separate this trend from chance. Comparing 0/3 at 30k with 2/3 at 500k gives a two-sided Fisher exact $p \approx 0.4$. Even at 500k, 2/3 is far below the 97.5% detection rate that the paper reports. A laptop with background processes and frequency scaling is a noisier measurement platform than the authors' setup, which lowers power for a small shift.
- **Practical consequence for an engineer.** A single run at a small sample size is not a reliable negative: an undetected leak at 30k does not show absence of leakage. A detection should also not be accepted from a single run, since the control produced one. Confidence should come from repeated runs and from larger samples, which is the paper's own recommendation (Section 6.1) to increase the sample size rather than the threshold α. As the paper states in Section 5.2, "no detection" means any leak is too subtle for this setup, not that the code is side-channel free.

6 Limitations and deviations

- **Three repeats per cell** give no usable estimate of detection or false-positive rates. Ten or more repeats at 30k, and more at larger sizes, would be needed.
- **Python re-implementation.** The authors' R implementation was not run. However, the Python version follows the published rule.

- **Fresh local measurements, not the authors' archived dataset.** Results depend on CPU scheduling, frequency scaling and system load on one machine. Runs were collected at different times, so slow drift between runs cannot be excluded.
- **Artificial channel.** The `shift-1%` channel is a controlled model. It does not capture the structure of the TLS measurements in the paper's real-world study.
- **Parameter choices.** $\alpha = 0.09$ is the value the paper uses for its ground-truth comparison. The paper's TLS study uses $\alpha = 0.009$, which would give fewer false alarms and lower power.
- RTLF does not localize the source of a leak. Here the source is known by construction, since the channel is artificial.